upGrad

Capstone Workflow: Sahayak Health AI

Introduction

This capstone project focuses on the development of Sahayak Health AI, an agentic decision-support assistant for community health workers (ASHA workers) in rural India. The system combines multi-agent orchestration (Google Agent Development Kit), staged clinical reasoning, an interactive follow-up loop, deterministic safety auditing, and a held-out final evaluation against declared clinical thresholds to convert a natural-language symptom description into exactly one care-seeking recommendation: WAIT, DOCTOR, or ER.

The solution follows a two-phase sequential-agent architecture in which an intake phase (symptom extraction → severity scoring → follow-up question) pauses for the health worker's answer, and a decision phase (triage decision → safe response formatting → safety audit) consumes that answer and may escalate the recommendation. The workflow progresses through business understanding, data understanding, baseline construction, full agent development, held-out evaluation, failure analysis, and train-only prompt recalibration with a before/after A/B comparison.

The final outcome is a locally runnable triage assistant (Google Colab with Gemini, or fully offline via Ollama) with a live two-page demonstration dashboard, a reproducible evaluation harness with saved artifacts, and a production-readiness gate that passes or fails on declared clinical criteria. Deliverables include notebooks, source modules, evaluation artifacts, the interactive dashboard, and project documentation.

Business Problem

India's rural primary-care system depends on roughly one million ASHA (Accredited Social Health Activist) workers — trained community members, not clinicians. Villagers bring them symptoms; the nearest doctor may be many kilometres away. The high-value decision is not *"what disease is this?"* but *"what is the right next step — wait at home, visit a clinic today, or go to emergency care now?"*

Getting this decision wrong is asymmetric:

- **Under-triage** (sending a sick patient home) can cost a life.
- **Over-triage** (escalating a self-limiting illness) wastes scarce clinic capacity and erodes trust, but harms no one directly.

Clinical triage practice encodes this asymmetry explicitly: the American College of Surgeons field-triage standard accepts up to 35–50% over-triage as the price of keeping under-triage at or below 5%. A useful triage assistant must therefore be evaluated on this asymmetric loss — raw accuracy alone is the wrong objective, and a system tuned only for accuracy can be clinically dangerous.

Potential applications include:

- ASHA / community-health-worker decision support
- Telehealth intake and pre-consultation routing
- Helpline (104/108) call triage assistance
- NGO field-clinic queue prioritisation

- Pharmacy walk-in guidance
- Health-camp screening support

The objective of this project is to develop a local agentic triage assistant that reads messy first-person symptom language, asks for missing information the way a clinician would, recommends exactly one care level with a calm plain-language explanation, never diagnoses or prescribes, and proves its safety posture with reproducible, artifact-backed evaluation.

Data Description

Primary dataset: gretelai/symptom_to_diagnosis (HuggingFace, Apache 2.0).

Property	Value
Train split	853 cases
Test split	212 cases (held out from all tuning)
Labels	22 diagnosis classes
Input (input_text)	First-person natural-language symptom description
Target (output_text)	Diagnosis label

Example input: *"I've been having a lot of pain in my neck and back. I've also been having trouble with my balance and coordination..."*

The diagnosis label is never shown to the user and never produced by the agent. It is used only to construct evaluation ground truth via the provided TRIAGE_MAP, which maps each of the 22 diagnoses to a care-seeking level:

- WAIT (11 diagnoses, ≈50% of cases): allergy, arthritis, cervical spondylosis, chicken pox, common cold, fungal infection, GERD, impetigo, migraine, psoriasis, varicose veins
- DOCTOR (10 diagnoses, ≈45%): dengue, malaria, typhoid, jaundice, drug reaction, bronchial asthma, hypertension, diabetes, urinary tract infection, peptic ulcer disease
- ER (1 diagnosis, ≈4%): pneumonia

Two properties of this mapping are taught explicitly because they shape every downstream metric:

1. **Dramatic language ≠ urgency.** Half the corpus is WAIT, and WAIT cases include vivid presentations (migraine with vomiting and light sensitivity; spondylosis with balance trouble). A system that escalates on language intensity will fail exactly where real triage fails.
2. **An irreducible label-noise floor exists.** Symptom text alone cannot always separate diagnoses that map to different levels (classic example: hepatitis-pattern presentations). Perfect accuracy is not attainable from this input, and learners must reason about that measurement ceiling.

Data split discipline: the **train-split** feeds EDA, the policy baseline, and prompt recalibration. The **test split** is reserved exclusively for the final live-agent evaluation and is never used for any form of tuning.

Overall Methodology and Learning Outcomes

System Architecture

The project follows a staged multi-agent architecture implemented using Python 3.11+, Jupyter Notebook, Google ADK (LlmAgent, SequentialAgent, Runner, session state), Gemini (Colab path) or Ollama (fully local path), FastAPI + Server-Sent Events (live dashboard), Pandas, and the HuggingFace datasets library.

A patient utterance is processed through a coordinator-style sequential workflow in which each agent has exactly one job and writes one key into session state, making every intermediate step inspectable and debuggable.

Phase A — Intake

- **Symptom Parser Agent** — extracts only the symptoms present in the text; no diagnosis, no inference beyond the words.
- **Severity Scorer Agent** — scores care-seeking urgency 1–5 against a calibrated clinical rubric (calibrated on the train-split; see Stage 4).
- **Follow-up Asker Agent** — for ambiguous severities (2–3) generates exactly one clarifying question a health worker can answer by observation; the pipeline then pauses and waits for the answer.

Phase B — Decision

- **Triage Decider Agent** — maps severity to WAIT/DOCTOR/ER; reads the follow-up answer and may escalate (never de-escalate) on red-flag content.
- **Response Formatter Agent** — produces a calm, action-first message for the worker, with a mandatory disclaimer; never diagnoses or prescribes.
- **Safety Evaluator Agent** — LLM auditor that checks rule compliance; its verdicts are cross-checked against a deterministic rule judge.

Why the Two-Phase Pause is Necessary: Real clinical conclusions need information the first message never contains — history, observations, vitals, labs. A triage agent cannot order tests, so its core skill is knowing *what information it lacks*: ask for what a health worker can observe, and escalate when the missing information is the kind only a clinic can provide. The two-phase design makes this a real architectural property, not a decorative question.

Evaluation Framework

All headline metrics are computed by deterministic code from saved per-case records — the LLM never grades itself. Each evaluation run writes a timestamped JSON artifact (config, per-case results, confusion matrix), and nothing is claimed without a file behind it.

Metric	Definition	Why it is reported
Under-triage rate	predicted level below true level	The safety metric; each step down is dangerous

ER-sent-home count	true ER predicted WAIT	The catastrophic error; tracked separately
Over-triage rate	predicted level above true level	The efficiency cost being traded for safety
Exact accuracy	predicted == true	Comparability with published baselines
Per-class recall + confusion matrix	standard	Locates *which* boundary fails
Weighted clinical cost	ER→WAIT=10, DOCTOR→WAIT=5, ER→DOCTOR=3, over-triage=1	Single asymmetric loss for optimisation gating
Safety-audit pass rate	deterministic judge verdicts	Compliance: no diagnosis, no prescription, disclaimer present
Judge agreement	LLM auditor vs deterministic judge	Teaches LLM-judge reliability measurement
Follow-up policy compliance	asked exactly when severity is 2–3	The asker must not delay an emergency or skip an ambiguous case
Follow-up relevance	question anchored to a symptom or red-flag domain	Deterministic, necessary-not-sufficient question-quality check
Loop-closure probe	answer-aware decider run twice per ambiguous case	Proves the answer changes the decision — a number, not a demo
Label fallback rate	unparseable decider output after JSON → regex → retry	Reliability: a triage system may never answer "?"

Acceptance thresholds are declared a priori from the clinical and ML literature, not fitted to results:

Criterion	Threshold	Anchor
Under-triage rate	≤ 5%	ACS field-triage standard
ER-sent-home	0	catastrophic-error policy
Over-triage rate	≤ 50% (target ≤ 35%)	ACS accepted band

Exact accuracy	≥ 60%	fine-tuned Llama-3.1-8B achieves 58.4% on real ESI triage (arXiv 2504.16273); frontier LLMs score 34–60% on 39k real ED cases (PMC12941425)
Human reference point	74% exact	nurse inter-rater agreement, Manchester Triage System (PMC6053287)
Follow-up policy compliance	≥ 90%	question-quality evaluation is an emerging axis: Q4Dx (Sci Rep 2025), evidence-elicitation (arXiv 2601.19773)
Follow-up relevance	≥ 90%	deterministic anchor check (symptom or red-flag domain)
Loop closure	deployed target compliance ≥ 80%, de-escalation = 0	red-flag answer must reach the mandated level via LLM decider + deterministic escalation floor; raw LLM compliance reported alongside
Label fallback rate	≤ 2%	reliability of structured output after recovery ladder

These anchors are themselves a learning outcome: learners discover that human experts agree on triage only ~74% of the time, so a 60–70% exact-accuracy agent with near-zero under-triage is a strong system, while a 90% claim on this task should trigger suspicion of leakage, not celebration.

Learning Outcomes

On completion, learners can:

- Decompose a decision problem into single-purpose agents with inspectable state
- Build a multi-turn information-gathering loop; construct ground truth from a proxy label and reason about its noise floor
- Select metrics that encode domain loss asymmetry
- Anchor acceptance criteria in published baselines
- Apply train-only prompt recalibration with a before/after A/B test
- And present an honest, artifact-backed evaluation story

Implementation Roadmap

The capstone is organised into a sequence of implementation notebooks that guide learners through the recommended development process. The notebooks are intended to support implementation and do not correspond directly to the assessment stages. Tasks from a single assessment stage may span multiple notebooks, and a notebook may contain tasks from multiple assessment stages. Learners should complete the notebooks in the recommended order to progressively build, evaluate, and refine the solution.

Notebooks	Files provided for reference	Tasks	Marks
adk_foundations.ipynb	utils.py, ADK docs	1.1, 1.2	9
data_understanding_and_baseline.ipynb + sahayak_starter.py	data_loader.py, utils.py	1.3, 1.4, 2.1, 2.2	21
agent_pipeline_development.ipynb	sahayak_tools.py, tests/test_sahayak_harness.py	1.5, 3.1, 3.2, 3.3, 3.4	29
agent_evaluation_and_optimisation.ipynb	eval_agent.py, demo_app.py	4.1, 4.2, 4.4, 4.4	41
		Total	**100**

Stage 1 – Business Objective, Data Understanding & Agent Foundations (24 marks)

Task 1.1 – Define the Triage Problem Scope [Deliverable: final_report.pdf] [3 marks]

- **Subtask 1.1.1 – Scope the problem:** Define the ASHA-worker persona, the three care levels (WAIT/DOCTOR/ER), the asymmetric-loss principle, explicit non-goals (no diagnosis, no prescription) and measurable success criteria.
    - *Expected output:* Problem definition with a persona and a thresholds table.

Task 1.2 – Learn ADK Through Small Examples [Deliverable: adk_foundations.ipynb] [6 marks]

- **Subtask 1.2.1 – Run ADK demos:** Build and execute ≥2 ADK demos, including a SequentialAgent chain with visible session state.
    - *Expected output:* Executed demo cells with printed session-state keys between agents.

- **Subtask 1.2.2 – Measure parallel speedup:** Compare the execution time of a ParallelAgent against the sequential chain and state the observed speedup.
    - *Expected output:* Timing comparison and calculated speedup factor.

Task 1.3 – Exploratory Data Analysis [Deliverable: data_understanding_and_baseline.ipynb] [4 marks]

- **Subtask 1.3.1 – Load and profile the dataset:** Load the dataset, inspect the first few records, compute text-length statistics, and visualise the distribution of the 22 diagnosis labels.
    - *Expected output:* EDA cells confirming the 853/212 train-test split and a diagnosis-label distribution chart.

Task 1.4 – Construct Ground Truth & Evaluation Sample [Deliverable: data_understanding_and_baseline.ipynb] [7 marks]

- **Subtask 1.4.1 – Review the diagnosis → triage mapping:** Review the provided TRIAGE_MAP in data_loader.py, validate it using sample records, and justify three clinically ambiguous mappings (for example, why jaundice maps to DOCTOR rather than ER).
    - *Expected output:* Spot-check output and written justifications for three edge cases.

- **Subtask 1.4.2 – Create the fixed evaluation sample:** Create a stratified n=50 sample using the provided random seed and clearly state the train/test split discipline.
    - *Expected output:* Sample-creation code cell and an explicit statement describing the train/test split strategy.

Task 1.5 – Design the Agent Architecture [Deliverable: final_report.pdf] [4 marks]

- **Subtask 1.5.1 – Specify the six-agent architecture:** Define the role, input keys, and output keys for each of the six agents, describe the two-phase pause mechanism, and explain the escalate-never-de-escalate rule.
    - *Expected output:* Architecture diagram and session-state flow table.

Stage 2 – Baseline System (10 marks)

Task 2.1 – Transparent Policy Baseline [Deliverable: data_understanding_and_baseline.ipynb] [6 marks]

- **Subtask 2.1.1 – Evaluate the baseline:** Implement the keyword/rule-based baseline and evaluate it on the fixed sample; report the overall accuracy and per-class recall for the WAIT, DOCTOR, and ER care levels.
    - *Expected output:* Baseline evaluation table showing overall accuracy and per-class recall for WAIT, DOCTOR, and ER.

- **Subtask 2.1.2 – Explain the failure mode:** Identify the dominant failure mode of the rule-based baseline and support your explanation with at least two representative misclassified examples (for example, rules failing to interpret natural language context, resulting in poor ER recall).
    - *Expected output:* Failure analysis describing the primary limitation of the baseline, supported by two misclassified examples.

Task 2.2 – First Two ADK Stages (Parser + Severity) [Deliverable: data_understanding_and_baseline.ipynb] [4 marks]

- **Subtask 2.2.1 – Build parser + severity scorer:** Implement the Symptom Parser Agent and Severity Scorer Agent as LlmAgents and evaluate them on a sample of 10 cases
    - *Expected output:* Trace table showing input symptoms, expected severity, and predicted severity for 10 evaluation cases.

Stage 3 – Solution v1: Full Six-Agent Pipeline (25 marks)

Task 3.1 – Follow-up Loop, Closed and Measured [Deliverable: agent_pipeline_development.ipynb] [8 marks]

- **Subtask 3.1.1 – Conditional follow-up:** Generate a follow-up question only for ambiguous severities (2–3) and achieve a follow-up policy compliance rate of at least 90%.
    - *Expected output:* Conditional follow-up implementation with followup_policy_compliance_rate ≥ 90%.
- **Subtask 3.1.2 – Close the loop:** Pause Phase A, accept the health worker's answer, and demonstrate that the additional information can change the triage decision. Achieve a loop_target_compliance_rate ≥ 80%.
    - *Expected output:* A logged example showing a WAIT→DOCTOR escalation driven by the follow-up response, along with the loop compliance metric.

Task 3.2 – Triage Decider & Safe Formatter [Deliverable: agent_pipeline_development.ipynb] [7 marks]

- **Subtask 3.2.1 – Escalation-only decider:** Escalate on red-flag answers and never de-escalate below the base rule (loop_de_escalation_count = 0).
    - *Expected output:* Working triage decider with deterministic escalation logic and loop_de_escalation_count = 0.
- **Subtask 3.2.2 – Safe response formatter**: Produce an action-first, calm message that includes the required disclaimer while avoiding diagnoses and treatment recommendations.
    - *Expected output:* Sample responses that successfully pass the safety evaluation.

Task 3.3 – Safety Evaluator & Deterministic Judge [Deliverable: agent_pipeline_development.ipynb, sahayak_starter.py] [6 marks]

- **Subtask 3.3.1 – Deterministic judge:** Implement all six compliance checks and generate PASS/FLAG verdicts for each evaluation case.
    - *Expected output:* Deterministic judge implementation with per-case compliance reports and identified violations.
- **Subtask 3.3.2 – Tests pass:** Execute the provided deterministic test suite successfully.
    - *Expected output:* Successful execution of all provided pytest tests.

Task 3.4 – End-to-End Demos [Deliverable: agent_pipeline_development.ipynb] [4 marks]

- **Subtask 3.4.1 – Four traced runs:** Execute and trace one WAIT case, one DOCTOR case, one ER case, and one example where the follow-up response changes the final recommendation end-to-end.
    - *Expected output:* Four fully traced workflow executions with agent interactions, tool calls, and outputs.

Stage 4 – Solution v2: Calibration, Evaluation & Communication (41 marks)

Task 4.1 – Failure Analysis on Solution v1 [Deliverable: agent_evaluation_and_optimisation.ipynb] [11 marks]

- **Subtask 4.1.1 – Confusion matrix:** Compute the confusion matrix and per-class recall on ≥50 cases.
    - *Expected output:* Confusion matrix and per-class recall values for all three care levels.

- **Subtask 4.1.2 – Diagnose the dominant error:** Analyse at least three incorrect predictions, identify the pipeline component responsible for the errors, and explain the dominant systematic failure mode.
    - *Expected output:* Failure analysis describing the primary source of errors (for example, Solution v1 over-escalates based on symptom intensity)

Task 4.2 – Calibrate Prompts on the Train-Split [Deliverable: agent_evaluation_and_optimisation.ipynb, sahayak_starter.py] [5 marks]

- **Subtask 4.2.1 – Recalibrate (train-only) with an A/B table:** Refine the severity-scoring instructions using only the training split and compare the original and revised prompts through an A/B evaluation.
    - *Expected output:* Before-and-after A/B comparison table together with the updated prompt instructions.

Task 4.3 – Final Held-Out Evaluation [Deliverable: agent_evaluation_and_optimisation.ipynb, final_report.pdf] [15 marks]

- **Subtask 4.3.1 – Run once, report against thresholds:** Execute the complete agent pipeline once on the held-out test set and compare every evaluation metric against its predefined acceptance threshold.
    - *Expected output:* acceptance table in notebook.

- **Subtask 4.3.2 – Safety targets:** Demonstrate that the final solution achieves an under-triage rate of at most 5% and zero ER-sent-home cases.
    - *Expected output:* Safety metric values visible in notebook.

- **Subtask 4.3.3 – Cite the anchors:** Support the reported thresholds by citing ACS field-triage guidance, nurse inter-rater agreement studies, and published LLM-triage baselines.

- *Expected output:* Cited thresholds included in the report.

Task 4.4 – Demonstration & Communication [Deliverable: final_report.pdf] [10 marks]

- **Subtask 4.4.1 – Two-page dashboard:** Present the interactive dashboard with a live custom-query page and a batch evaluation page showing the confusion matrix and per-class recall.
  - *Expected output:* Running demo_app.py demonstrating both dashboard views.

- **Subtask 4.4.2 – Known-limits section:** Explain the system's performance ceiling, dataset label noise, and the fact that the solution is not intended to function as a medical device.
  - *Expected output:* A limits section covering all three in the report.

- **Subtask 4.4.3 – Narrate for a non-technical reviewer:** Present one WAIT, one DOCTOR, and one ER scenario in language that can be easily understood by a non-technical reviewer.
  - *Expected output:* Clear demonstration narrative suitable for a non-technical audience.

Assessment Note: The performance thresholds provided in this workflow are recommended benchmarks rather than mandatory grading cut-offs. As you may use different local LLMs, model sizes, and inference configurations, the exact metric values may vary. You will not be penalised solely for failing to achieve these benchmark values. Assessment will primarily focus on the correctness of the implementation, agent workflow design, evaluation methodology, and the evidence provided to justify the observed results.

Deliverables

#	File	Type / Description
1	adk_foundations.ipynb	Foundation notebook containing guided ADK exercises. Learners complete the required concept cells and implementation tasks.
2	data_understanding_and_baseline.ipynb	Implementation notebook covering data understanding, exploratory data analysis, ground-truth construction, and the transparent policy baseline.
3	agent_pipeline_development.ipynb	Implementation notebook for developing the complete six-agent pipeline, follow-up loop, response generation, and safety evaluation.
4	agent_evaluation_and_optimisation.ipynb	Implementation notebook covering failure analysis, prompt calibration, final evaluation, and performance reporting.

5	sahayak_starter.py	Learner implementation file containing the core pipeline functions to be completed during the capstone, including run_policy_triage() and safety_evaluator_agent().
6	final_report.pdf	Final project report describing the methodology, evaluation results, failure analysis, known limitations, references, and recommendations.

Acceptance criteria: Every notebook should execute sequentially from top to bottom in either Google Colab (Gemini) or a local Python environment (Ollama). All reported metrics must be generated within the notebooks and visible as notebook outputs. The final report should reference the corresponding notebook outputs for every reported metric. The complete solution should execute without requiring a local GPU.

Tools, Libraries, and Technology Stack

The capstone is designed to be implemented using open-source tools and can be executed either on Google Colab using Gemini models or entirely on a local machine using Ollama. The recommended technology stack is outlined below.

Programming Language: Python 3.11+

Development Environment

- Jupyter Notebook
- Google Colab (Gemini execution path)
- Local Python Environment (Ollama execution path)
- Visual Studio Code (optional)

Data Handling and Analysis

- Pandas
- NumPy
- Hugging Face Datasets

Agent Development Framework

- Google Agent Development Kit (ADK)

- LlmAgent
- SequentialAgent
- ParallelAgent
- Runner
- Session State Management

Large Language Models: Learners may use one of the following execution paths.

- **Cloud-Based Path:** Gemini models through Google Colab
- **Fully Local Path:** Ollama-compatible models running locally

Application Development

- FastAPI
- Server-Sent Events (SSE)

Testing and Evaluation

- Pytest
- JSON
- Collections
- Datetime
- Matplotlib
- Seaborn
- Scikit-learn (for confusion matrices and evaluation visualisations)

Visualisation and Demonstration

- Matplotlib
- Seaborn
- FastAPI Dashboard
- Interactive Web Interface for Live Demonstration

Implementation Guidance

The capstone is organised as a staged implementation exercise. Learners are encouraged to complete tasks in the recommended sequence and validate outputs incrementally rather than implementing the entire system at once.

Stage 1 – Business Understanding, Data Understanding, and Agent Foundations

- Carefully read the business problem before beginning implementation.
- Understand why clinical triage is an asymmetric decision problem and why under-triage is significantly more harmful than over-triage.
- Explore the dataset thoroughly before developing any agents.
- Review the provided TRIAGE_MAP and understand how diagnosis labels are mapped to care-seeking recommendations.
- Complete the ADK exercises and become comfortable with session state and agent communication before building the full pipeline.
- Create the fixed evaluation sample exactly once using the provided random seed and maintain strict train/test separation throughout the project.

Stage 2 – Baseline System

- Begin with the transparent policy baseline before introducing LLM-based agents.
- Evaluate the limitations of rule-based systems and document their failure modes.
- Implement the Symptom Parser Agent and Severity Scorer Agent independently and verify intermediate outputs.
- Inspect session state after each agent execution and confirm that each agent writes exactly the expected output.

Stage 3 – Solution v1: Full Six-Agent Pipeline

- Build one agent at a time and validate each component independently before chaining them together.
- Ensure that follow-up questions are generated only for ambiguous severities.
- Implement the escalation rule carefully. Follow-up information may increase urgency but must never reduce urgency below the base recommendation.
- Verify that responses:

  - recommend exactly one care-seeking level,
  - remain calm and action oriented,
  - include the required disclaimer,
  - never diagnose, and
  - never prescribe treatment.
- Execute all deterministic tests before proceeding to final evaluation.

Stage 4 – Solution v2: Calibration, Evaluation, and Communication

- Perform failure analysis before modifying prompts or instructions.
- Conduct prompt recalibration only on the training split.
- Never use held-out test cases during calibration or prompt tuning.
- Run the final evaluation only after all implementation components are complete.
- Generate and save all evaluation artefacts, including confusion matrices, per-case outputs, and metric summaries.
- Use the final report to communicate both strengths and limitations of the system.

General Recommendations

- Implement and test incrementally.
- Save intermediate artefacts frequently.
- Use deterministic configurations wherever possible to improve reproducibility.
- Keep prompts concise and task-specific.
- Document assumptions and implementation decisions throughout the project.
- Prioritise safety metrics over exact accuracy when interpreting results.

Common Mistakes and Troubleshooting

1. Treating Diagnosis Prediction as the Objective

- **Issue:** Attempting to predict or generate diagnoses.

- **Resolution:** The system must produce exactly one care-seeking recommendation: WAIT, DOCTOR, or ER. Diagnosis labels are used only for evaluation and ground-truth construction.

2. Using the Test Split During Calibration

- **Issue:** Using held-out cases while rewriting prompts or adjusting instructions.
- **Resolution:** Prompt recalibration must be performed exclusively on the training split. The test split is reserved for final evaluation.

3. Generating Follow-Up Questions for Every Case

- **Issue:** Asking follow-up questions unconditionally.
- **Resolution:** Follow-up questions should only be generated for ambiguous severity levels (2–3).

4. Allowing De-Escalation After Follow-Up

- **Issue:** A red-flag answer reduces the original recommendation.
- **Resolution:** Follow-up information may increase urgency but must never reduce urgency below the base recommendation.

5. Agents Overwriting Session State

- **Issue:** Multiple agents modify the same session keys.
- **Resolution:** Each agent should have one clearly defined responsibility and write only its designated outputs.

6. Responses that Diagnose or Prescribe

- **Issue:** The final response predicts diseases or recommends treatments.
- **Resolution:** Responses should:
    - recommend one care-seeking level,
    - explain the next action calmly,
    - include the disclaimer, and
    - avoid diagnosis and treatment recommendations.

7. Low Follow-Up Policy Compliance

- **Issue:** Follow-up compliance metrics remain below the required threshold.
- **Possible Causes:**
    - Severity boundaries are poorly defined.
    - Ambiguous cases are not correctly identified.
    - Follow-up triggers are implemented inconsistently.
- **Resolution:** Inspect severity outputs and verify the conditions under which questions are generated.

8. High Under-Triage Rate

- **Issue:** The system frequently predicts a lower urgency than the ground truth.
- **Possible Causes:**
    - Symptom severity rules are too conservative.
    - Red-flag escalation logic is incomplete.
    - Important symptom information is not extracted.
- **Resolution:** Perform failure analysis and revise prompt instructions using only the training split.

9. Inconsistent Outputs Across Runs

- **Issue:** Identical inputs produce different recommendations.
- **Possible Causes:**
    - Non-deterministic model settings.
    - Prompts containing ambiguous instructions.
- **Resolution:** Use deterministic configurations and keep prompts precise and task specific.

10. Failed Deterministic Tests

- **Issue:** Provided tests do not pass.
- **Possible Causes:**
    - Incorrect function signatures.
    - Missing session keys.

- Invalid output formats.

- **Resolution:** Review the starter file specifications carefully and verify intermediate outputs before running the full pipeline.

11. Dashboard Does Not Display Results

- **Issue:** Live demonstrations fail to update correctly.
- **Possible Causes:**
  - Evaluation artefacts were not generated.
  - FastAPI server was not started correctly.
  - Required files were not saved in the expected locations.
- **Resolution:** Re-run the evaluation pipeline, regenerate artefacts, and verify application configuration before launching the dashboard.

12. Focusing Solely on Exact Accuracy

- **Issue:** Optimising only for accuracy while ignoring safety metrics.
- **Resolution:** Clinical triage is an asymmetric decision problem. Under-triage and catastrophic errors are more important than maximising exact accuracy. Final evaluation should therefore prioritise safety metrics and explain the trade-offs observed.